Chapitre 4 : Structuration d'une page HTML5 et Positionnement avec CSS

Pour concevoir des interfaces web modernes, réactives et professionnelles, il est désormais nécessaire de prendre le contrôle complet de l'organisation spatiale de la page.

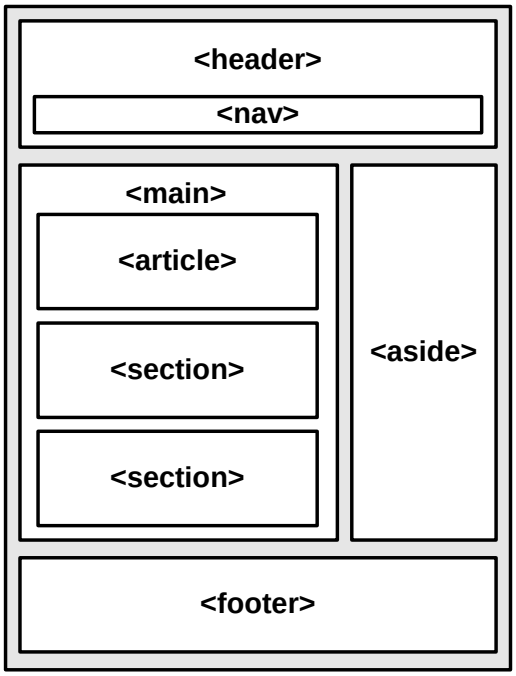
Ce chapitre est dédié à la structuration et au positionnement avancé. Il aborde l'utilisation des balises structurantes sémantiques avant d'explorer les deux piliers majeurs de la mise en page moderne : le modèle **Flexbox** pour l'alignement fluide à une dimension, et le modèle **CSS Grid** pour l'organisation bidirectionnelle en grilles complexes.

Enfin, le chapitre traite comment adapter dynamiquement ces dispositions à toutes les tailles d'écrans (t aux principes du *Responsive Web Design*.

I. Les balises structurantes

Plusieurs balises sont offertes par HTML5 pour délimiter les différentes zones qui constituent la page web :

- **<header>** : en-tête ;
- **<footer>** : pied de page ;
- <**nav>** : liens principaux de navigation
- **<main>** : contenu principal de la page
- **<section>** : section de page ;
- **<aside>** : informations complémentaires
- **<article>** : article indépendant.



<header> : l'en-tête

La plupart des sites web possèdent en général un en-tête, appelé *header* en anglais. On y trouve le plus souvent un logo, une bannière, le slogan du site…

Ces informations sont placées à l'intérieur de la balise <header> :

```
<header>
 <!-- Placez ici le contenu de l'en-tête de votre page -->
</header>
```

Remarques

- L'en-tête peut contenir : images, liens, textes…
- Il peut y avoir plusieurs en-têtes dans une page. Si celle-ci est découpée en plusieurs sections, chaque section peut en effet avoir son propre <header>.

<footer> : le pied de page

Le pied de page se trouve en général tout en bas du document. On y trouve des informations comme des liens de contact, le nom de l'auteur, les mentions légales, etc.

```
<footer>
  <!-- Placez ici le contenu du pied de page -->
</footer>
```

<nav> : principaux liens de navigation

La balise **<nav>** regroupe tous les principaux liens de navigation du site. On y place par exemple le menu principal du site.

Généralement, le menu est réalisé sous forme de liste à puces à l'intérieur de la balise **<nav>** :

```
<nav>
        <ul>
                <li><a href="index.html">Accueil</a></li>
                <li><a href="forum.html">Forum</a></li>
                <li><a href="contact.html">Contact</a></li>
        </ul>
</nav>
```

<main> : contenu principal de la page

Elément unique dans une page, il regroupe et enveloppe le contenu principal de la page

```
<main>
  <h1>Titre principal</h1>
  <p>Contenu central...</p>
  <section> .... </section>
</main>
```

<section> : une section de page

La balise <section> sert à regrouper des contenus en fonction de leur thématique. Elle englobe généralement une portion du contenu au centre de la page.

```
<section>
  <h1>Ma section de page</h1>
  <p>Bla bla bla bla</p>
</section>
```

Remarque :

Chaque section peut avoir son titre de niveau 1 (<h1>), de même que l'en-tête peut contenir un titre <h1> lui aussi.

Chacun de ces blocs étant indépendant des autres, il n'est pas illogique de retrouver plusieurs titres <h1> dans le code de la page web. On a ainsi « Le titre <h1> d'une première section et pareil pour une autre section, etc.

<aside> : informations complémentaires

La balise <aside> est conçue pour contenir des informations complémentaires au document que l'on visualise. Ces informations sont généralement placées sur le côté (bien que ce ne soit pas une obligation).

```
<aside>
  <!-- Placez ici des informations complémentaires -->
</aside>
```

Remarque **:** Il peut y avoir plusieurs blocs <aside> dans la page.

<article> : un article indépendant

La balise <article> sert à englober une portion généralement autonome de la page. C'est une partie de la page qui pourrait ainsi être reprise sur un autre site. C'est le cas par exemple des actualités (articles de journaux ou de blogs).

```
<article>
  <h1>Un article </h1>
  <p>Le texte de l'article </p>
</article>
```

Exemple

```
<body>
    <header>
        <h1>Le journal du village</h1>
        <nav>
            <ul>
                <li><a href="Accueil.html">Accueil</a></li>
                <li><a href="contenu.html">Accès au contenu</a></li>
                <li><a href="contact.html">Contact</a></li>
            </ul>
        </nav>
    </header>

    <main>
        <aside>
```

```
            <p><strong>Articles rédigés par Mohamed Said</strong></p>
        </aside>
        <section>
            <h2>Les news de la semaine</h2>
            <article>
                <h3>Un lion dans la forêt</h3>
                <p>Ali Foulen, un chasseur du village, a aperçu un lion
dans la forêt...</p>
            </article>
            <article>
                <h3>Une chute d'astéroïde</h3>
                <p>Hier après-midi, un astéroïde géant est tombé ...</p>
            </article>
        </section>
    </main>

    <footer>
        <p>Copyright XXXX - Tous droits réservés</p>
    </footer>
</body>
```

Aperçu :



***NB** : Les nouvelles balises ne sont reconnues par Internet Explorer que depuis sa version 9 (IE9).*

➢ Les balises universelles

En plus des balises vues précédemment il existe deux balises pour créer une zone :

- **… :** balise en ligne *(inline)*, donc une balise dans un paragraphe ou plutôt dans une balise dite « bloc » ; elle s'utilise au sein d'un bloc pour une modification ponctuelle ;

- ** … ** : balise de bloc *(block)*, un bloc est créé, il y a un retour à la ligne automatique.

Le principal intérêt de ces balises est que l'on peut leur appliquer une class (ou un id) pour le CSS quand aucune autre balise ne convient.

Exemple

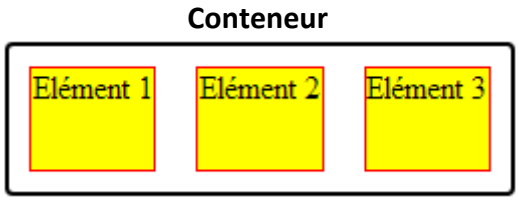
`<div>0000000<div>111111</div>222222</div>`	0000000 111111 222222
`<p><span>0000000<span>111111</span>222222</span></p>`	0000000111111222222

II. Positionnement dans CSS avec le modèle de boîte flexible : Flexbox

« Flexible Box Layout Module » connu sous le nom « Flexbox » est une spécification CSS3 du W3C qui définit un nouveau modèle de boîte et de positionnement qui apporte des solutions au ***Responsive Webdesign*** en offrant un agencement automatique et adaptatif à l'intérieur d'un composant.

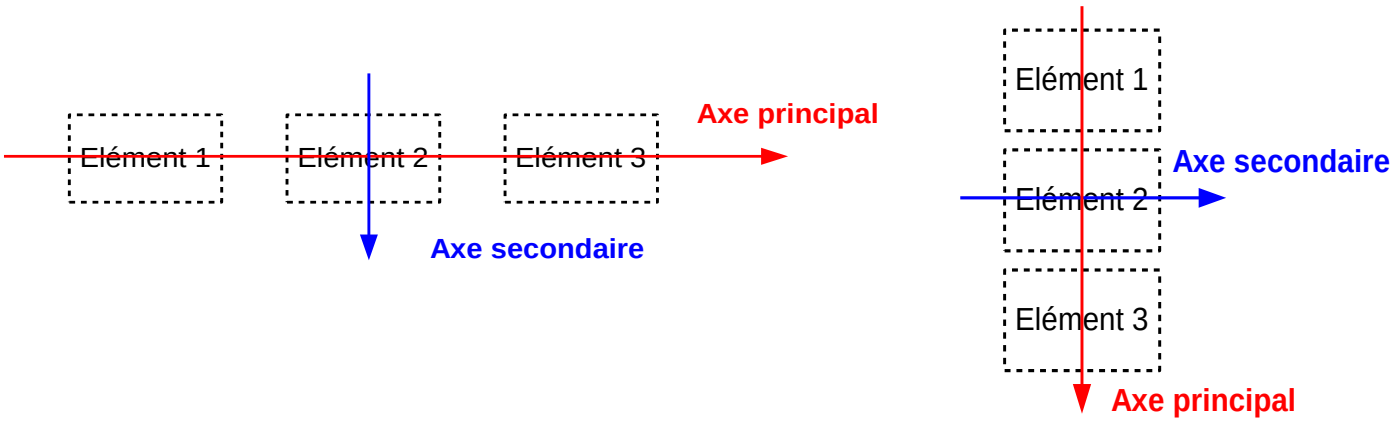
II.1. Les bases

Flexbox est un module (et non une propriété) qui a ses propres propriétés. Il est basé sur le principe d'un **conteneur** (élément parent appelé flex-container) et d'**éléments** contenus (éléments enfants nommés flex-items).



Les éléments sont organisés soit horizontalement (par défaut), soit verticalement. On définit ainsi l'axe principal et l'axe secondaire (qui lui est perpendiculaire).

- Si les éléments sont alignés horizontalement, l'axe principal est horizontal et l'axe secondaire est vertical
- Si les éléments sont alignés verticalement, l'axe principal est vertical et l'axe secondaire est horizontal



Déclaration d'un conteneur flex:

```
display : flex;
```

Exemple :

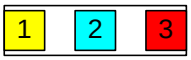
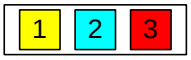
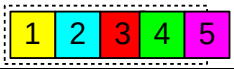
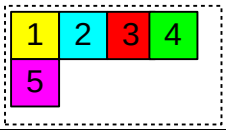
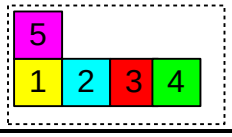
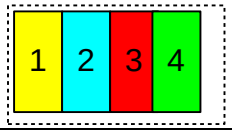
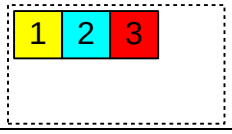
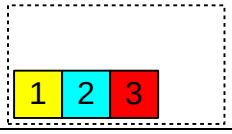
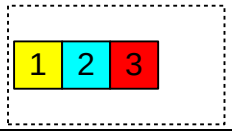
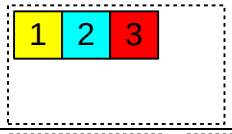
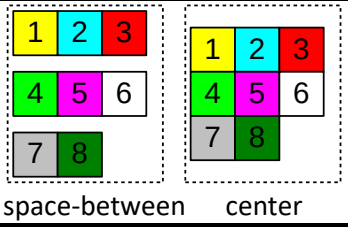
Code HTML	Code CSS	Résultat
  Elément 1 Elément 2 Elément 3 	.conteneur {**display:flex;** } .c1 { width:100px; border: 2px solid red; }	Elément 1 Elément 2 Elément 3

II.2. Principales propriétés

Le modèle flexbox offre des propriétés pouvant être appliquées sur le conteneur ou sur un élément particulier afin de lui attribuer un positionnement particulier.

II.2.1. Propriétés qui s'appliquent au conteneur

Propriétés	Valeurs possibles	Représentation
flex-direction : Permet d'aligner les éléments du conteneur horizontalement ou verticalement avec la possibilité de les inverser.	row	1 2 3
	row-reverse	3 2 1
	column	1 2 3
	column-reverse	3 2 1
justify-content : Permet d'aligner les éléments sur l'axe principal	flex-start	1 2 3
	flex-end	1 2 3
	center	1 2 3

	space-between	
	space-around	
flex-wrap : Donne la possibilité que les blocs aillent à la ligne lorsqu'ils n'ont pas de place.	nowrap	
	wrap	
	Wrap-reverse	
align-items : Permet d'aligner les items sur l'axe secondaire	stretch	
	flex-start	
	flex-end	
	center	
	baseline	
align-content : Définit comment les lignes d'un flexbox sont répartis	flex-start | flex-end | center | stretch |space-between |space-around	

Remarque:

Démonstrations en ligne des principales propriétés

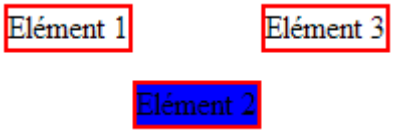
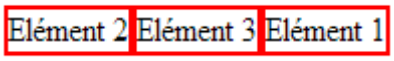
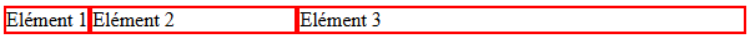
https://codepen.io/enxaneta/full/adLPwv

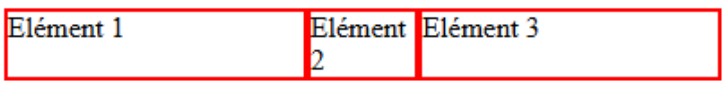
II.2.2. Propriétés qui s'appliquent à un élément

Pour spécifier le style d'un élément particulier du conteneur, il peut être sélectionné avec : **nth-child (***position***).**

Soit le code html suivant :

```
<div id="conteneur">
        <div class="c1"> Elément 1 </div>
        <div class="c1"> Elément 2 </div>
        <div class="c1"> Elément 3 </div>
</div>
```

Propriétés	Exemple
align-self: auto | flex-start | flex-end | center | baseline | stretch Effectue une exception pour l'alignement d'un élément sur l'axe secondaire	#conteneur { display: flex; align-items: flex-start;} .c1 {border :2px solid red;} .c1:**nth-child(2)** { background-color: blue; **align-self**: flex-end; } 
order : nombre Modifie l'ordre des éléments. En indiquant un nombre, les éléments sont triés du petit au plus grand	#conteneur { display: flex;} .c1 {border :2px solid red;} .c1:nth-child(1) { **order:3;** } .c1:nth-child(2) { **order:1;** } .c1:nth-child(3) { **order:2;** } 
flex-grow : détermine la capacité d'un élément flexible à **s'étendre** pour occuper l'espace disponible dans le container. Elle prend des valeurs positives (par défaut 0)	#conteneur { display: flex; } .c1 {border :2px solid red;} .c1:nth-child(2){ **flex-grow: 1**; } .c1:nth-child(3){ **flex-grow: 3**; } 

flex-shrink : définit le facteur de rétrécissement d'un élément flexible.
Valeur par défaut : 1,
0 : empêche l'élément de se rétrécir
	#conteneur { display: flex; **width:400px;** }
	.c1 {border :2px solid red; **width: 200px**;}
	.c1:nth-child(2){
	flex-shrink: 4;
	}
	 

flex : nombre propriété raccourcie qui définit la capacité d'un élément flexible à modifier ses dimensions afin de remplir l'espace disponible de son conteneur. Les éléments flexibles peuvent être étirés ou réduits pour utiliser un	#conteneur { display: flex;} .c1 {border :2px solid red;} .c1:nth-child(1) { **flex:2;** } .c1:nth-child(2) { **flex:1;** } 

espace proportionnel à leur coefficient de grossissement ou de rétrécissement afin de ne pas dépasser d'un conteneur.	

III. Positionnement dans CSS avec le modèle de grille : Grid

CSS Grid permet un contrôle **bidimensionnel** du positionnement, bien plus puissant que les méthodes traditionnelles.

La grille permet d'aligner des éléments sous forme de colonnes et de lignes avec la possibilité de disposer de régions nommées.

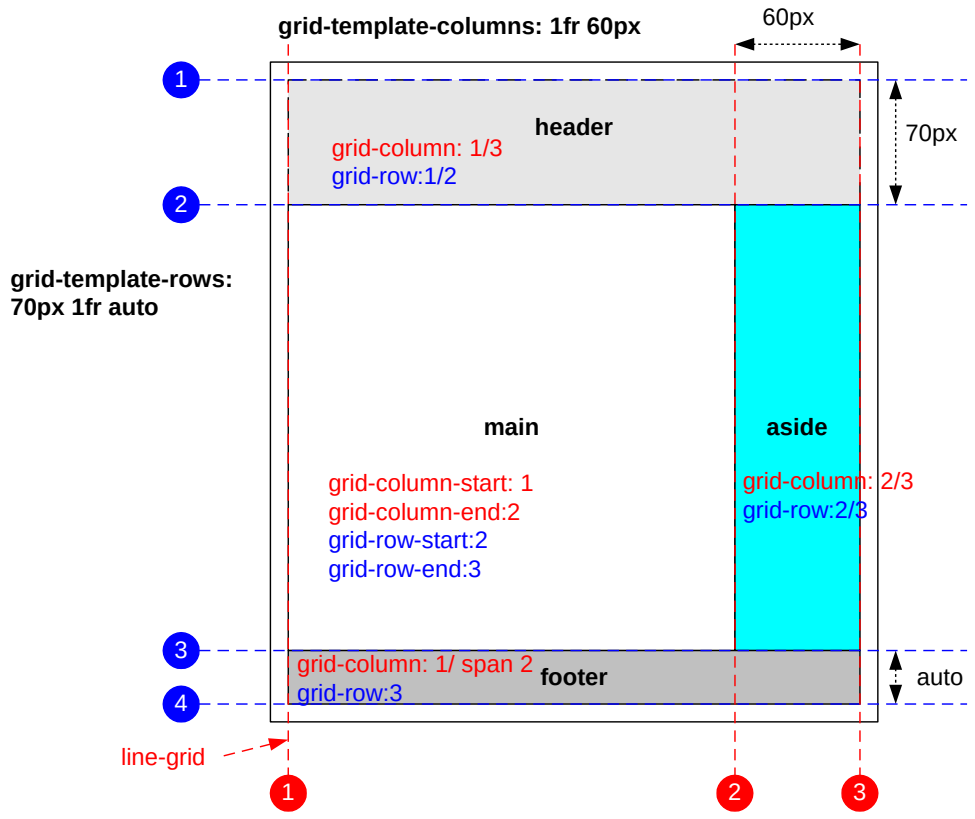
Déclaration d'un conteneur grid:

```
display : grid;
```

III.1. Concepts de base des grilles

Une grille est un ensemble de lignes horizontales et verticales qui se croisent : les premières définissant les rangées, et les secondes les colonnes.

Le schéma suivant illustre les propriétés de base.



Principales propriétés :

Propriété	Description	Exemple
display: grid	Définit le conteneur comme grille	display: grid;
grid-template-columns	Définit les colonnes de la grille	grid-template-columns: 1fr 2fr 50px;
grid-template-rows	Définit les lignes de la grille	grid-template-rows: 100px auto 200px;
grid-column-start	Début de placement sur les colonnes	grid-column-start: 2;
grid-column-end	Fin de placement sur les colonnes	grid-column-end: 4;
grid-row-start	Début de placement sur les lignes	grid-row-start: 1;
grid-row-end	Fin de placement sur les lignes	grid-row-end: 3;
grid-column	Raccourci pour column-start/end	grid-column: 1 / 3;
grid-row	Raccourci pour row-start/end	grid-row: 1 / span 2 ;
grid-auto-rows	Taille des lignes qui sont créées automatiquement quand des éléments sont placés en dehors de la grille définie.	grid-auto-rows: 200px; */* Les lignes supplémentaires auront 200px */*
grid-auto-columns	Taille des colonnes générées en dehors de la grille prédéfinie	grid-auto-columns: 1fr;
gap	Espacement entre les cellules (row-gap + column-gap)	gap: 5px;

- **Valeurs des propriétés : grid-template-columns et grid-template-rows**

Les colonnes et les lignes peuvent avoir :

- ✓ Des tailles fixes : px, em, rem
- ✓ Des tailles relatives : %, vw, vh
- ✓ Des unités flexibles : fr (fraction d'espace disponible)
- ✓ Une taille automatique : auto (s'adapte au contenu)
- ✓ Des fonctions de taille : minmax(valeur_min, valeur_max) et fit-content(valeur)

Remarque :

La fonction repeat peut être utilisée pour répéter un modèle:

grid-template-columns: 50px **repeat**(3, 1fr 40px);

→ 7 colonnes : 50px 1fr 40px 1fr 40px 1fr 40px

- **Grid responsive avec auto-fit et auto-fill**

Pour obtenir une grille complètement responsive, il est possible de combiner les propriétés *repeat*, *minmax* avec ***auto-fit*** ou ***auto-fill***:

- ✓ Propriété **auto-fit :** permet de créer un nombre dynamique de pistes (colonnes ou lignes) pour s'adapter à l'espace disponible selon une taille minimale (souvent défini avec minmax()).
- ✓ Propriété **auto-fill** : La grille **crée autant de colonnes que possible** selon une taille minimale même si elles sont vides

Exemple :

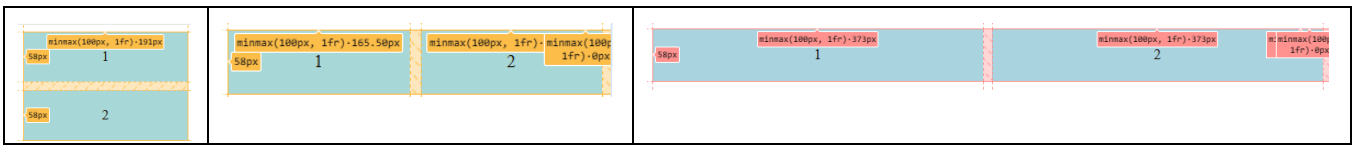
Soit le code initial suivant

Code HTML	Code CSS

`<div class="container">`
`  <div class="item">1</div>`
`  <div class="item">2</div>`
`</div>`
	`.container {display: grid;  gap: 10px;  }`
	`.item { background: lightblue;`
	`padding: 20px; text-align: center;}`

1/ En ajoutant à la classe container cette définition :

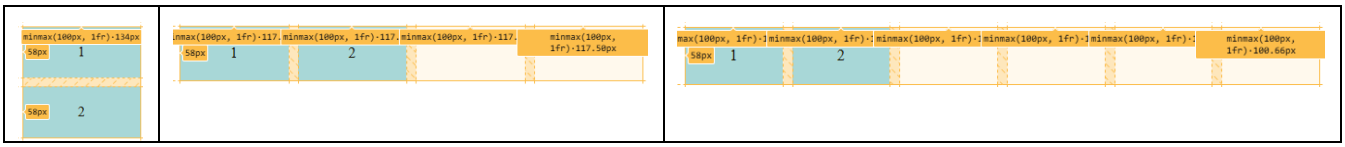
grid-template-columns: repeat(**auto-fit**, minmax(100px,1fr));



La grille étire les colonnes existantes pour remplir l'espace disponible

2/ En remplaçant auto-fit par auto-fill

grid-template-columns: repeat(**auto-fill**, minmax(100px,1fr));



La grille crée un maximum de colonnes même si elles sont vides

- **Valeurs des propriétés : grid-column et grid-row**

Le même principe s'applique sur les lignes et les colonnes. Les valeurs de grid-column peuvent être :

début/fin	grid-column: 2 / 4; */* Débute à la line-grid 2 et s'étend jusqu'à la line-grid 4 */*
span <nombre> (nombre de colonnes)	grid-column: 1 / span 3; */* Commence à la line-grid 1 et couvre 3 colonnes */*

<nombre>	grid-column: 2; */* Commence à la line-grid 2 et occupe 1 colonne */*
auto Comportement par défaut	grid-column: auto; */* Début et fin automatiques : L'élément occupe 1 seule colonne */*
nomLigneDébut/ **nomLigneFin** Si les lignes de la grille sont nommées	.container { grid-template-columns: [col1] 100px [col2] 100px [col3]; } */* Les noms des line-grid doivent être délimités par des crochets[]*/* .item { grid-column: col1 / col3; } */* Utilise les noms des line-grid*/*

Exemple :

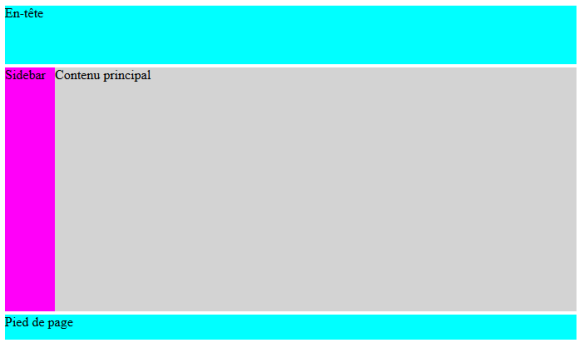
On considère, pour la suite, le code de HTML suivant :

Code HTML	Code CSS

`<div class="grid-container">`
`  <header>En-tête</header>`
`  <main>`
`  Contenu principal`
`  </main>`
`  <aside>Sidebar</aside>`
`  <footer>Pied de page</footer>`
`</div>`
	`header, footer`
	`{background-color: aqua;}`
	`main {background-color: lightgray;}`
	`aside {background-color: magenta;}`
	 

On positionne avec CSS Grid comme suit :

Code CSS	Aperçu du rendu

`/* Avec le positionnement */`
`.grid-container {`
`  display: grid;`
`  grid-template-columns: 60px 1fr;`
`  grid-template-rows:70px 1fr 30px;`
`  height: 400px;`
`  row-gap: 4px; }`
`header { background-color: aqua;`
`  grid-column-start: 1;`
`  grid-column-end: 3;`
`  grid-row-start: 1;`
`  grid-row-end: 2;}`
`main { background-color: lightgray;`
`  grid-column: 2/3;`
`  grid-row: 2/3;}`
`aside { background-color: magenta;`
`  grid-column: 1;`
`  grid-row: 2; }`
`footer { background-color: aqua;`
`  grid-column: 1/3;`
	

`grid-row: 3; }`	

III.2. Positionnement avec des zones nommées

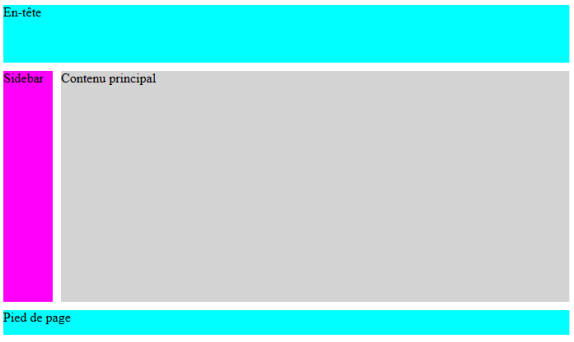
Il est possible de structurer la page visuellement à travers des zones nommées moyennant les propriétés grid-template-areas et grid-area.

Propriété	Description	Exemple
grid-template-areas	Définit, au niveau du conteneur, des zones nommées dans la grille	grid-template-areas: "header header" "sidebar main";
grid-area	Définit, dans un élément, un raccourci ou une référence à une zone nommée	grid-area: sidebar;

Exemple :

On considère le même code HTML de l'exemple précédent.

Code CSS	Aperçu du rendu

`.grid-container {`
`display: grid;`
`grid-template-columns: 60px 1fr;`
`grid-template-rows: 70px 1fr 30px;`
`grid-template-areas: "header header"`
`"aside main"`
`"footer footer";`
`height: 400px;`
`gap: 10px;}`
`header {`
`background-color: aqua;`
`grid-area: header; }`
`main {`
`background-color: lightgray;`
`grid-area: main; }`
`aside {`
`background-color: magenta;`
`grid-area: aside; }`
	

footer { background-color: aqua; grid-area: footer; }	

III.3. Alignement de la grille et des cellules

Il est possible de contrôler l'alignement dans CSS Grid à travers des propriétés qui ciblent soit toute la grille, soit les éléments/cellules de la grille.

Principales propriétés :

Propriété	Description	Valeurs possibles
justify-content	Aligne l'ensemble de la grille le long de l'axe en ligne (horizontal) lorsque la taille totale de la grille est plus petite que le conteneur.	start, end, center, stretch, space-around, space-between, space-evenly
align-content	Aligne l'ensemble de la grille le long de l'axe en colonne (vertical) lorsque la taille totale de la grille est plus petite que le conteneur.	start, end, center, stretch, space-around, space-between, space-evenly.
place-content	Propriété raccourcie pour align-content et justify-content. Si une seule valeur est donnée, elle s'applique aux deux axes	place-content: <align-content> <justify-content> ;
justify-items	Aligne les éléments le long de l'axe en ligne (horizontal) à l'intérieur de leur cellule de grille.	start, end, center, stretch (par défaut).
align-items	Aligne les éléments le long de l'axe en colonne (vertical) à l'intérieur de leur cellule de grille.	start, end, center, baseline, stretch (par défaut)
place-items	Propriété raccourcie pour align-items et justify-items. Si une seule valeur est donnée, elle s'applique aux deux axes	place-items: <align-items> <justify-items>
justify-self	Aligne le contenu spécifique d'une cellule selon l'axe horizontal	start, end, center, stretch (par défaut), baseline.
align-self	Aligne le contenu spécifique d'une cellule selon l'axe vertical	auto (valeur par défaut), start, end, center, baseline, stretch
place-self	Propriété raccourcie pour align-self et justify-self.	place-self: <align-self> <justify-self>

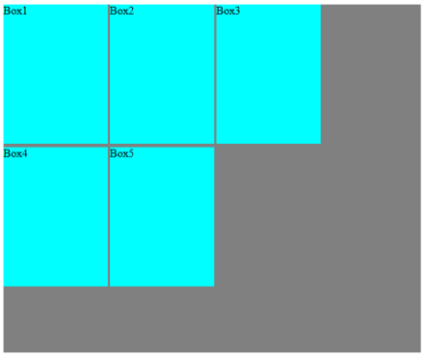
Exemples :

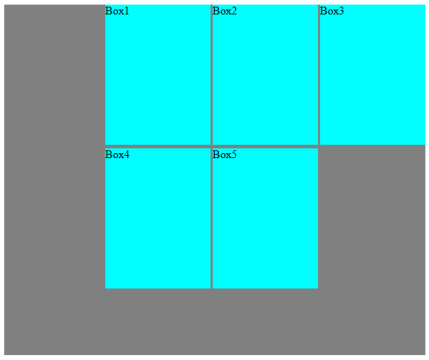
On considère ce code HTML suivant

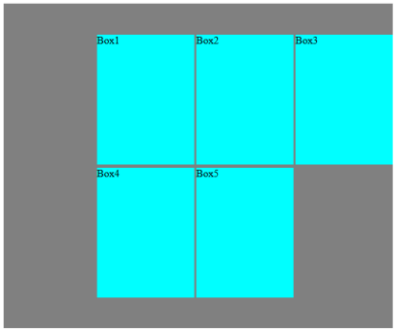
```
<div class="grid-container">
```

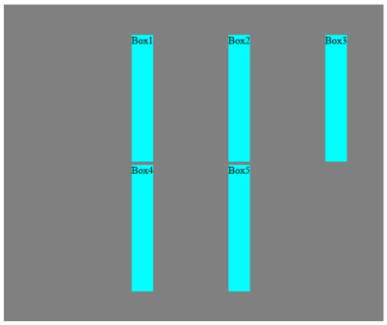
```
    <div class="item"> Box1</div>
    <div class="item"> Box2</div>
    <div class="item"> Box3</div>
    <div class="item"> Box4</div>
    <div class="item"> Box5</div>
  </div>
```

Code CSS	Rendu

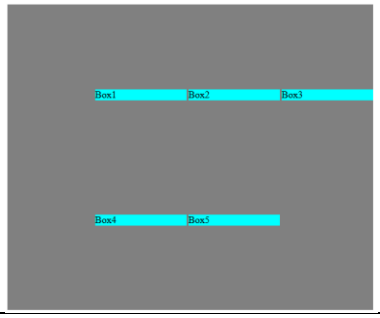
`.grid-container {`
`background-color: gray;`
`display: grid;`
`grid-template-columns: repeat(3,150px);`
`grid-template-rows: repeat(2, 200px);`
`width: 600px;`
`height: 500px;`
`gap: 5px 3px;`
`}`
`.item {background-color: aqua;}`
	

`.grid-container {`
`background-color: gray;`
`display: grid;`
`grid-template-columns: repeat(3, 150px);`
`grid-template-rows: repeat(2, 200px);`
`…`
`justify-content: end;`
`}`
	

`.grid-container {`
`background-color: gray;`
`display: grid;`
`grid-template-columns: repeat(3, 150px);`
`grid-template-rows: repeat(2, 200px);`
`…`
`justify-content: end;`
`align-content: center;`
`}`
	

`.grid-container {`
`display: grid;`
`grid-template-columns: repeat(3, 150px);`
`grid-template-rows: repeat(2, 200px);`
`…`
`place-content: center end;`
`justify-items: center;`
`}`
	

```
.grid-container {
display: grid;
grid-template-columns: repeat(3, 150px);
grid-template-rows: repeat(2, 200px);
…
place-content: center end;
align-items: center;
}
```



IV. Le responsive design avec les Media Query

Le responsive design permet d'adapter l'affichage d'un site à la taille de l'écran : ordinateur, tablette, smartphone…
Pour adapter une page aux écrans mobiles, il faut ajouter à l'entête de la page, il faut

1. **Ajouter la balise `<meta name="viewport">` dans le <head> :**

Sans cette balise, les navigateurs mobiles simulent un écran de PC et le rendu sera minuscule ou déformé.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2. **Définir les media queries en CSS :**

Les **Media Queries** en CSS sont l'outil principal pour cela. Ce sont des règles qui indiquent quand on doit appliquer des propriétés CSS.

Syntaxe générale :

```
@media (condition) {
   /* règles CSS à appliquer */
}
```

Exemples :

```
/* 1. STYLES DE BASE : Appliqués par défaut (Mobile First) */
body {
    background-color: aqua;                  /* Couleur par défaut (mobile) */
    font-size: 16px;                         /* Taille de police par défaut */
    font-family: Arial, Helvetica, sans-serif;/* Hérité par tous les écrans */
}

/* 2. ÉCRANS MOYENS : Tablettes (>= 601px) */
@media (min-width: 601px) {
    body {
        background-color: lightgreen; /* On surcharge uniquement la couleur */
    }
}
```

```css
/* 3. ÉCRANS LARGES : Ordinateurs (>= 1025px) */
@media (min-width: 1025px) {
    body {
        background-color: yellow;        /* On surcharge la couleur */
        font-size: 18px;                 /* On augmente la taille de police */
    }
}
```

Remarque :

L'approche la plus courante consiste à **commencer par la version mobile** (*mobile first*) puis élargir les styles avec des min-width.

Ce chapitre a posé le socle indispensable à la création d'interfaces web professionnelles : une architecture sémantique claire, combinée à la puissance de Flexbox et de CSS Grid sous le prisme du Responsive Web Design.